

**Universidad Autónoma de
Yucatán**

Facultad de Matemáticas

Estructuras de Datos

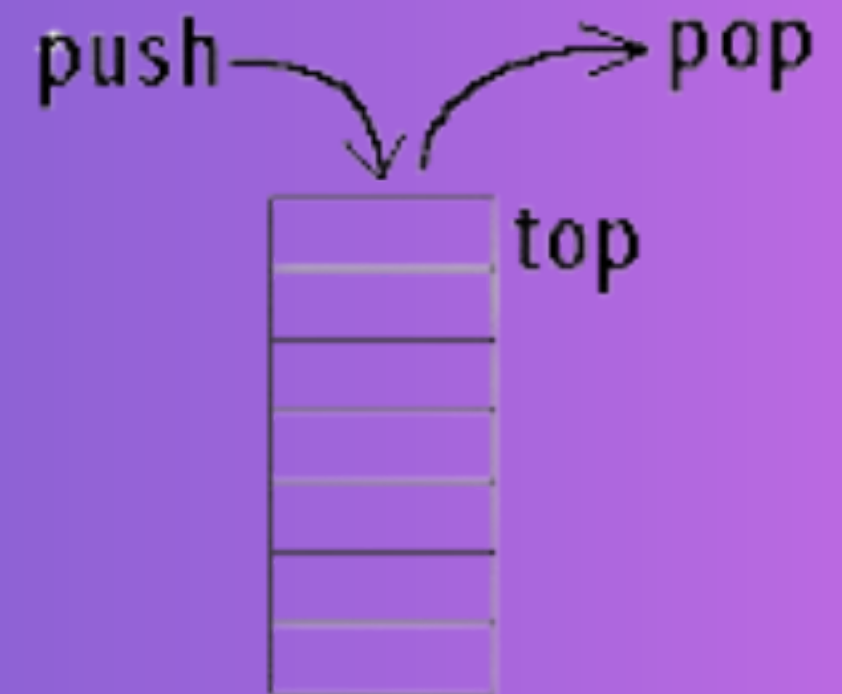
Unidad 1.



Implementación de pilas utilizando listas ligadas en C

Equipo N

- Carlos Emmanuel Romero Pot
- Aldrin Enrique Novelo Góngora
- Erick Ricardo Vega Nolasco



GENERALIDADES DE LA LIBRERIA RELACIONADA

PILA

Funciones:

- Push (node *list, int number)
- Pop (node*list)
- top()
- PopFromTop(Struct **node hd)
- PushFromTop(Struct **node hd, int x, int y)
- void push(struct node** head_ref, int new_data)
- struct node* push(struct node* head, int new_data)

LISTAS LIGADAS

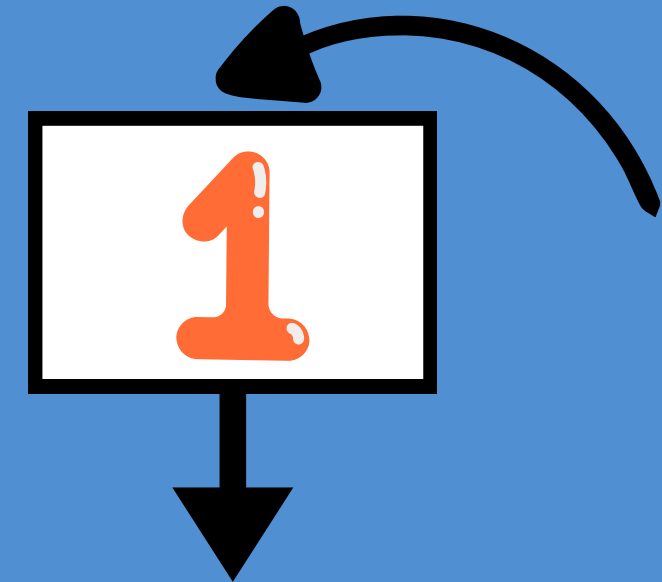
Funciones:

- free_list(node *list)
- void printList(struct node *node)
- x2

ERROR 1: ERROR DE SEGMENTACION EN LA FUNCIÓN

CASO A RESOLVER

El programa intenta simular el proceso de una pila utilizando listas simplemente ligadas en el lenguaje C por medio de punteros pero existen un error en la función `popFromTop()` cuando existe solo un nodo, lo que provoca una falla de segmentación.



CODIGO FUENTE

```
#include <stdio.h>
#include <stdlib.h>

struct node{
    int xposition;
    int yposition;
    struct node* next;
};

void pushToTop(struct node** hd, int x, int y){
    struct node* curr= *hd;
    struct node* prev=NULL;

    while(curr!=NULL){
        prev=curr;
        curr= curr->next;
    }
    struct node* ptr= (struct node*)malloc(sizeof(struct node));
    ptr->xposition=x;
    ptr->yposition=y;
    ptr->next=curr;

    if(prev==NULL){
        *hd= ptr;}
    else{
        prev->next=ptr;
    }
}
```

Estructura que define un nodo para una lista enlazada.

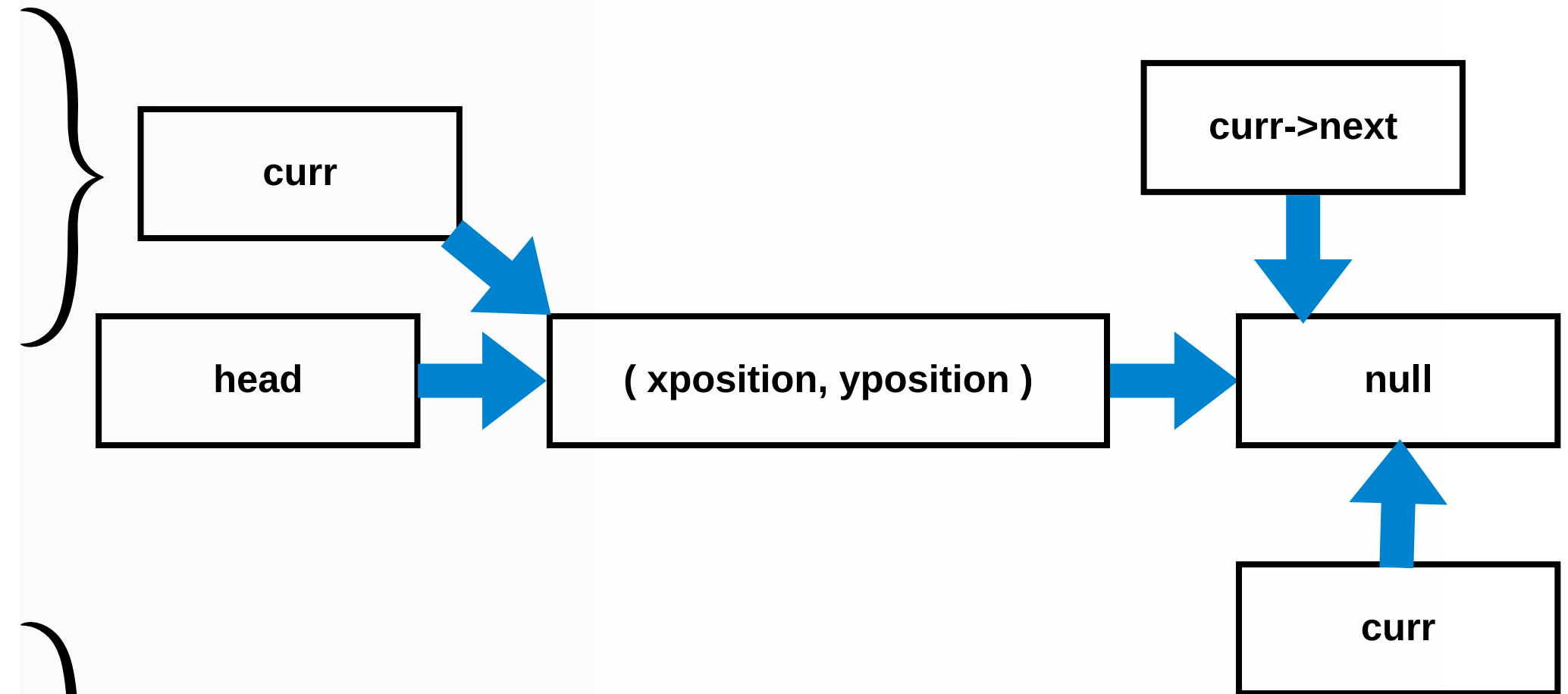


Agregar un nodo a la pila

CODIGO FUENTE

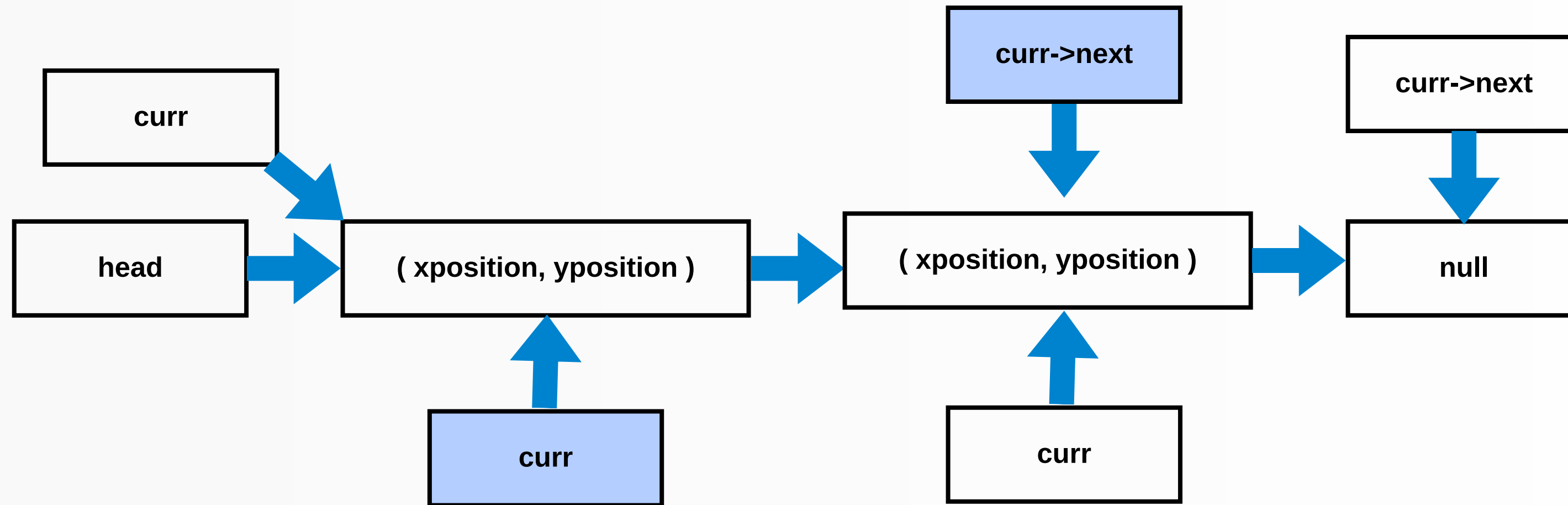
```
void popFromTop(struct node** hd ){  
  
    struct node* curr= *hd;  
    struct node* prev=NULL;  
    while ( curr->next !=NULL) {  
        prev=curr;  
        curr=curr->next;  
    }  
  
    free(curr);  
    prev->next= NULL;  
}
```

ERROR



Como curr->next es null el ciclo while no se realiza por lo que prev sigue siendo igual a null

CODIGO FUENTE



Aquí como si entra en el while curr si esta apuntando al penúltimo y si tiene `->next` que seria null

CODIGO FUENTE

ERROR

```
C:\Program Files (x86)\Zinjal\ × + v
Lista después de inserciones:
(10, 20) -> NULL

<< El programa ha finalizado: codigo de salida: 3221225477 >>
<< Presione enter para cerrar esta ventana >>
```

Si imprime la pila
pero no elimina el
elemento

```
// Insertar algunos nodos
pushToTop(&head, 10, 20);
```

```
printf("Lista después de inserciones:\n");
printList(head);
```

```
// Eliminar nodos
popFromTop(&head);
printf("Lista después de eliminar un nodo:\n");
printList(head);
```


CODIGO FUENTE

SOLUCIÓN

```
// Función para eliminar el último nodo de la lista  
void popFromTop(struct node** hd) {
```

```
    if (*hd == NULL) {  
        printf("La lista está vacía\n");  
        return;  
    }
```

```
    struct node* curr = *hd;  
    struct node* prev = NULL;
```

```
    // Recorrer la lista hasta el último nodo  
    while (curr->next != NULL) {  
        prev = curr;  
        curr = curr->next;  
    }
```

```
    // Si solo hay un nodo, actualizar la cabeza
```

```
    if (prev == NULL) {  
        free(curr);  
        *hd = NULL;  
    } else {  
        free(curr);  
        prev->next = NULL;  
    }
```

```
}
```

Agregar if par que si head esta vacía significa que esta vacía la lista y si prev = null head sea null

SOLUCIÓN

```
C:\Program Files (x86)\Zinjal\ X + v
Lista después de inserciones:
(10, 20) -> NULL
Lista después de eliminar un nodo:
NULL
La lista está vacía

<< El programa ha finalizado: codigo de salida: 0 >>
<< Presione enter para cerrar esta ventana >>|
```

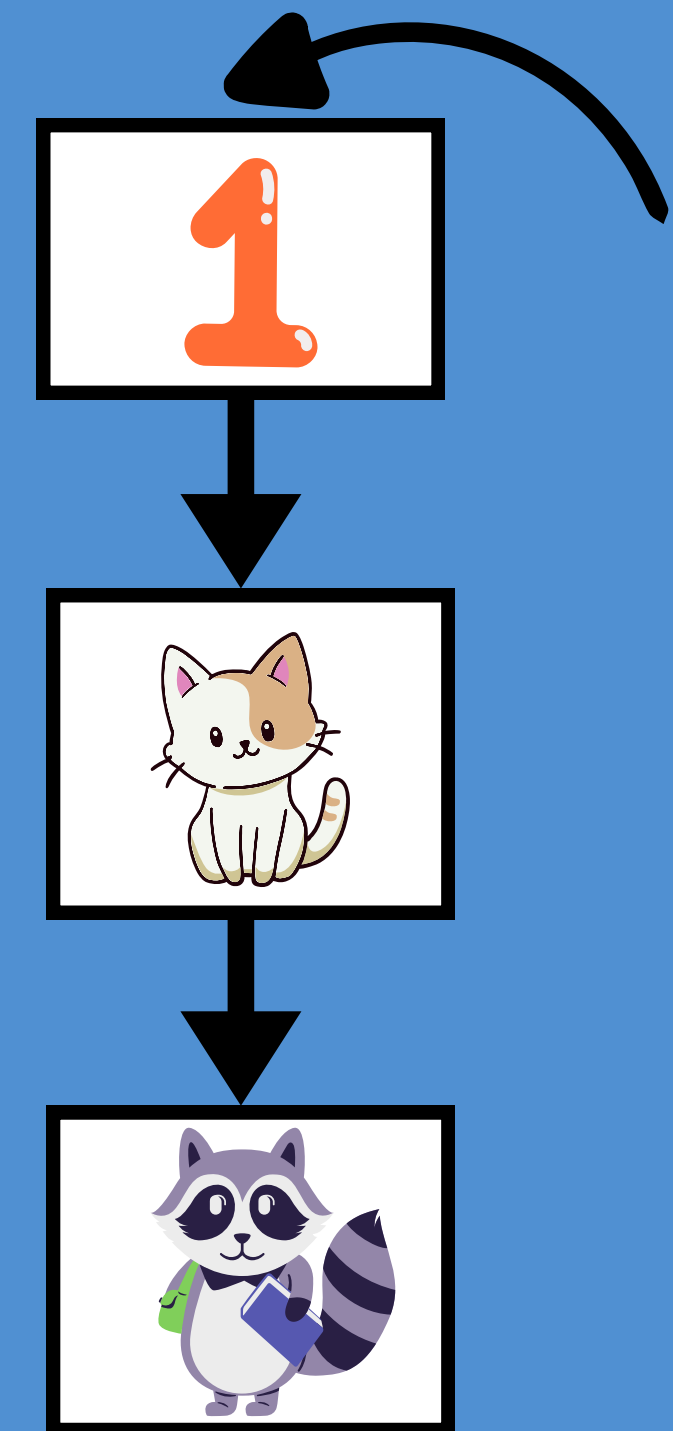
Ya imprime que la lista está vacía

ERROR 2: PASO DE PUNTERO POR VALOR EN LUGAR DE REFERENCIA

CASO A RESOLVER

El programa intenta simular el proceso de una pila utilizando listas simplemente ligadas en el lenguaje C por medio de apuntadores e implementando los metodos Push(), Pop() y free_list() .

Hay varios errores en el código que impiden que la lista enlazada funcione correctamente como una pila (LIFO).



CODIGO FUENTE

Objetivo: meter 3 numeros a la pila, vaciar la pila e imprimir los resultados.

```
int main(void)
{

    node *list = NULL;

    push(&list, 1);
    push(&list, 2);
    push(&list, 4);

    while(list != NULL) {

        printf(" %i ", pop(&list));
    }

    printf("%i\n", list->number);

    free_list(list);

    return 0;
}
```

} Declaramos la lista vacía y le añadimos elementos

} Sacamos los elementos de la pila

CODIGO FUENTE

```
// Dynamic size
typedef struct node
{
    int number;
    struct node *next;
}
node;
```

```
// Prototypes
void push(node *list, int number);
int pop(node *list);
void free_list(node *list);
```

Estructura auto
referenciada del
nodo

```
void push(node *list, int number)
{
    node *temp = malloc(sizeof(node));
    if (temp == NULL)
    {
        puts("Could not allocate memeory for element");
        return;
    }

    temp->number = number;
    temp->next = list;

    list = temp;
    // BUG: Value getting assigned here
}
```

```
int pop(node *list)
{
    if (list == NULL)
        return 0;

    node *temp = list;

    int number = list->number;
    list = list->next;

    free(temp);

    return number;
}
```

```
//Free memory of list
void free_list(node *list)
{
    while (list != NULL)
    {
        node *temp = list->next;
        free(list);
        list = temp;
    }
}
```


RESULTADOS

Expectativa

```
"D:\LENGUAJES DE PROGRAM" × + v  
4 2 1  
Process returned 0 (0x0)    execution time : 0.931 s  
Press any key to continue.
```

Realidad

```
"D:\LENGUAJES DE PROGRAM" × + v  
Process returned -1073741819 (0xC0000005)    execution time : 1.798  
Press any key to continue.
```

pq?



SOLUCION DEL ERROR

El problema ocurre en el paso de la lista como argumento en las funciones `push()` y `pop()` ya que la lista se pasa **por valor** y no por **referencia**.

El parámetro dentro de la función `push` es una **copia** del puntero original y cualquier cambio hecho no afecta la lista declarada en el `main()`.

🔍 ¿Dónde está el error?



```
typedef struct node
{
    int number;
    struct node *next;
}
node;

// Prototypes
void push(node *list, int number);
int pop(node *list);
void free_list(node *list);

int main(void)
{
    // Making empty linked list
    node *list = NULL;

    // Adding elements to list
    push(&list, 1);
    push(&list, 2);
    push(&list, 4);
}
```

CORRIGIENDO EL CÓDIGO

```
// Prototypes
void push(node **list, int number);
int pop(node **list);
void free_list(node *list);
```

```
void push(node **list, int number)
{
    node *temp = malloc(sizeof(node));
    if (temp == NULL)
    {
        puts("Could not allocate memory for element");
        return;
    }
```

```
    temp->number = number;
    temp->next = *list;
```

```
    *list = temp; |
}
```

Para modificar la lista en main(), debemos usar un puntero doble, para que los cambios se reflejen fuera de la función.

Ahora podemos acceder al puntero del main y cambiar su dirección de memoria.

CORRIGIENDO EL CÓDIGO

```
int pop(node **list)
{
    if (*list == NULL)
        return 0;

    node *temp = *list;

    int number = temp->number;
    *list = temp->next;

    free(temp);

    return number;
}
```

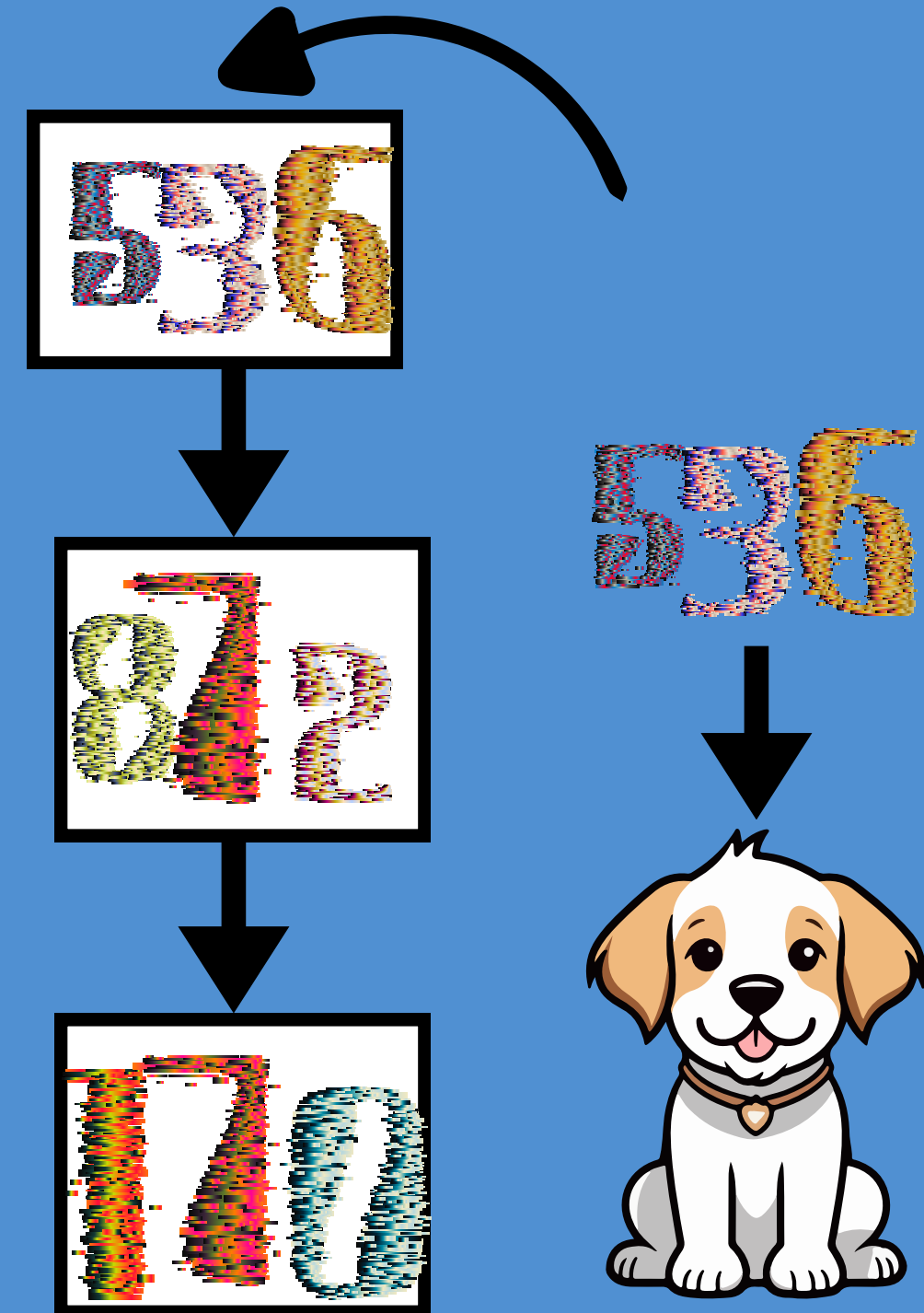
En la función Pop() también debemos de recibir un puntero a la lista del main (**list).

Dentro de la función debemos de referenciar el puntero a lista de manera correcta.

ERROR 3 AL IMPLEMENTACION DE PUSH() SIN USAR DOBLE APUNTADOR

CASO A RESOLVER

El código original implementa la función `push()` utilizando un doble apuntador, lo cual funciona correctamente. Sin embargo, al intentar implementarla con un solo apuntador, `push()` dejó de funcionar adecuadamente.



CODIGO FUENTE

Push() usando punteros dobles

```
#include <stdio.h>
#include <stdlib.h>
```

```
struct node
{
    int data;
    struct node *next;
};
```

La estructura del nodo contiene un dato y un puntero que apunta a otro nodo.

```
void push(struct node** head_ref, int new_data)
{
    struct node* new_node = (struct node*) malloc(sizeof(struct node));
    new_node->data = new_data;
    new_node->next = (*head_ref);
    (*head_ref) = new_node;
}
```

- La función push() tendrá como entrada un puntero que apunta a otro puntero (head) y una variable int que representa el dato.
- Crea un nuevo nodo y reserva un bloque de memoria.
- Asigna un valor al nuevo nodo.
- El nuevo nodo apunta a la cabeza de la lista.
- Actualiza la cabeza de la lista.

CODIGO FUENTE

Push() usando punteros dobles

```
void printList(struct node *node)
{
    while (node != NULL)
    {
        printf(" %d ", node->data);
        node = node->next;
    }
}

int main()
{
    struct node* head = NULL;
    push(&head, 6);
    push(&head, 7);
    push(&head, 1);
    push(&head, 4);
    printf("\n pila: ");
    printList(head);
    getchar();
    return 0;
}
```

La función imprimir recorre la lista ligada desde la cabeza hasta NULL.

- Crea un puntero a un nodo (head) y lo inicializa en NULL.
- Recibe como entradas la dirección del inicio y un dato.

CODIGO FUENTE

Push() usando punteros simple

```
#include <stdio.h>
#include <stdlib.h>

struct node
[ {
    int data;
    struct node *next;
- };

struct node* push(struct node* head, int new_data)
[ {
    struct node* new_node = (struct node*) malloc(sizeof(struct node));
    new_node->data = new_data;
    new_node->next = head;
    head = new_node;
    return head;
- }
```



- Es una función de tipo struct node*.
- Tiene como entrada un puntero al primer nodo (head) y un dato entero (new_data).
- Crea un nuevo nodo y reserva memoria para él con malloc().
- Asigna el valor de new_data al nuevo nodo.
- El nuevo nodo apunta a la cabeza de la lista.
- Retorna head, que ahora apunta al nuevo nodo.

CODIGO FUENTE

Push() usando punteros simple

```
void printList(struct node *node)
{
    while (node != NULL)
    {
        printf(" %d ", node->data);
        node = node->next;
    }
}
```

La función imprimir recorre la lista ligada desde la cabeza hasta NULL.

```
int main()
{
    struct node* head = NULL;

    head= push(&head, 6);
    head=push(&head, 7);
    head=push(&head, 1);
    head=push(&head, 4);
    printf("\n pila: ");
    printList(head);
    getchar();
    return 0;
}
```

- Crea un puntero a un nodo (head) y lo inicializa en NULL.
- Cada vez que se usa push(), se actualiza head con la nueva dirección del nodo insertado.
- push() recibe como entradas una dirección de memoria (head) y un dato entero.
- Llama a la función printList() pasando head como entrada para imprimir la lista.

Resultados

Push() usando punteros dobles

```
pila: 4 1 7 6 |
```

Push() usando punteros simple

```
pila: 4 -95953424 -95953264 |
```

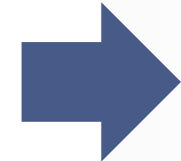
SOLUCION DEL ERROR

```
int main()
{
    struct node* head = NULL;

    head= push(&head, 6);
    head=push(&head, 7);
    head=push(&head, 1);
    head=push(&head, 4);
    printf("\n pila: ");
    printList(head);
    getchar();
    return 0;
}
```



La función push() espera un struct node*, pero se le está pasando &head, que es un struct node**. Esto genera un error de tipo, ya que push() solo necesita un puntero simple



```
int main()
{
    struct node* head = NULL;
    head = push(head, 6);
    head = push(head, 7);
    head = push(head, 1);
    head = push(head, 4);

    printf("\n pila:");
    printList(head);

    return 0;
}
```

Resultado:

```
pila: 4 1 7 6
```


EJEMPLO DE CREACIÓN PROPIA

Una implementación correcta de una pila usando listas simplemente enlazadas con funciones agregadas: imprimirPila(), size(), isEmpty()...

```
typedef struct nodo
{
    int num;
    struct nodo *siguiente;
    int cont;
} nodo;

void push(int num, nodo **tope ){

    //Creacion de un nuevo nodo
    nodo *nodoNuevo = malloc(sizeof(nodo));

    if(nodoNuevo == NULL){
        printf("La memoria no pudo ser asignada correctamente");
        return;
    }

    nodoNuevo->num = num;
    nodoNuevo-> siguiente = *tope;

    if (*tope == NULL) {
        nodoNuevo->cont = 1; // Si la pila esta vacia, el contador comienza en 0
    } else {
        nodoNuevo->cont = (*tope)->cont + 1; // Si no, se incrementa el contador del nodo superior
    }

    *tope = nodoNuevo;
}
```

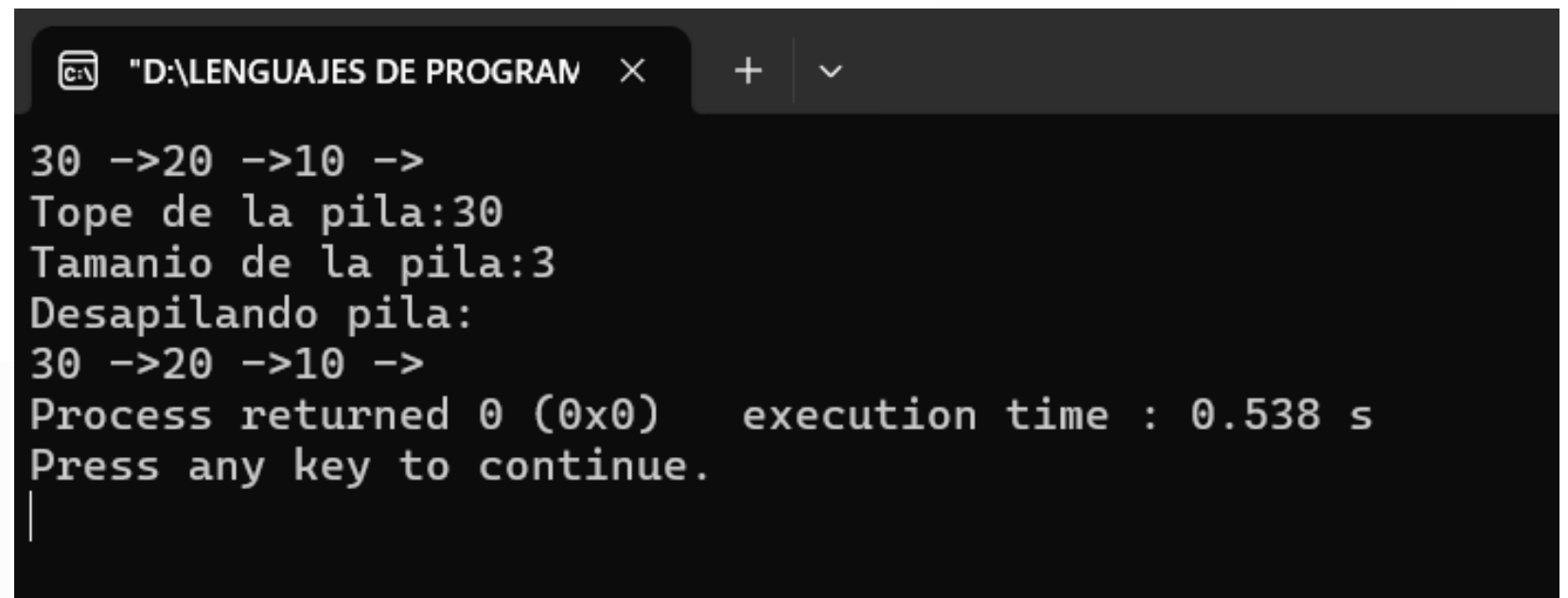
EJEMPLO DE CREACIÓN PROPIA

```
int pop(nodo **tope) {  
  
    if(*tope == NULL) {  
        printf("La lista esta vacia")    ;  
        return 0;  
    }  
  
    nodo *nodoTemp = *tope;  
  
    int eliminado = nodoTemp->num;  
  
    *tope = nodoTemp->siguiente;  
  
    if (*tope != NULL) {  
        (*tope)->cont = (*tope)->cont - 1;  
    }  
  
    free(nodoTemp);  
  
    return eliminado;  
  
}
```

```
void imprimirPila(nodo *tope) {  
  
    while(tope != NULL) {  
        printf("%i ->", tope->num);  
        tope = tope->siguiente;  
    }  
}  
  
int size(nodo *tope) {  
    if(tope == NULL) {  
        return 0;  
    }  
  
    int n = tope->cont;  
    return n;  
}  
  
int isEmpty(nodo **tope) {  
    int n = 0;  
    if(*tope != NULL) {  
        n = 1;  
    }  
    return n;  
}
```

EJEMPLO DE CREACIÓN PROPIA

```
int main() {  
  
    //Creacion de una pila aplicando la solución del error 2: paso por referencia y no por valor  
    nodo *pila = NULL;  
  
    push(10, &pila);  
    push(20, &pila);  
    push(30, &pila);  
  
    imprimirPila(pila);  
  
    printf("\nTope de la pila:%i\n", pila->num);  
    printf("Tamano de la pila:%i\n", size(pila));  
  
    printf("Desapilando pila: \n");  
    while(isEmpty(&pila) != 0){  
  
        printf("%i ->", pop(&pila));  
    }  
    return 0;  
}
```



```
"D:\LENGUAJES DE PROGRAM" x + v  
30 ->20 ->10 ->  
Tope de la pila:30  
Tamano de la pila:3  
Desapilando pila:  
30 ->20 ->10 ->  
Process returned 0 (0x0)    execution time : 0.538 s  
Press any key to continue.  
|
```

REFERENCIAS

Alcore. (2016, September 30). Pop function in linked list stack results in segmentation fault C [Discussion post]. Stack Overflow. <https://stackoverflow.com/questions/39784052/pop-function-in-linked-list-stack-results-in-segmentation-fault-c>

dexter. (2015, June 23). Linked list implementation in C without using double pointer [Discussion post]. Stack Overflow. <https://stackoverflow.com/questions/31010856/linked-list-implementation-in-c-without-using-double-pointer>

Randhawa, A. (2021, February 23). Incorrect implementation of stack datatype using linked list in C. Stack Overflow. <https://stackoverflow.com/questions/66342528>